

Entrega Final — Implantação PortfolioHUB + IA GEMINI

Aluno: Vitor Gomes Muzzi **Curso:** Engenharia de Software — CEUB **Data de Entrega:** 14/06/2026
Repositório: <https://github.com/VitorMuzzi/Bootcamp-Final> **Site em Produção:** <https://vitormuzzi.github.io/Bootcamp-Final/>

1. Planejamento da Implantação

1.1 Visão Geral do Projeto

O **PortfolioHUB** é uma plataforma web centralizada para exibição e gerenciamento de projetos e portfólios digitais. O objetivo é criar uma presença online profissional que integre:

- Site de portfólio pessoal (frontend estático)
- Repositório centralizado no GitHub para armazenamento e versionamento
- Integração em tempo real com a API do GitHub para exibição dinâmica de repositórios
- Políticas de segurança e controle de acesso configuradas no GitHub

1.2 Uso do Google GEMINI como Guia de Implantação

O Google Gemini foi utilizado em todas as etapas do projeto como assistente de planejamento e validação de decisões técnicas. As principais contribuições foram:

Planejamento da arquitetura:

Prompt utilizado: "Quero criar um portfólio web estático hospedado no GitHub Pages com integração à API do GitHub. Quais são as melhores práticas de arquitetura para esse tipo de projeto?"

O Gemini recomendou a estrutura de arquivo único (HTML + CSS + JS separados), hospedagem via GitHub Pages e uso da GitHub REST API v3 para dados dinâmicos — abordagem adotada integralmente no projeto.

Validação de segurança:

Prompt utilizado: "Quais configurações de segurança devo ativar no GitHub para um repositório público de portfólio?"

O Gemini listou: Branch Protection Rules, Dependabot, Secret Scanning, 2FA e SECURITY.md — todas implementadas conforme descrito na Seção 3.

1.3 Plano de Implantação (Fases)

Fase	Etapas	Status
1	Criação do repositório no GitHub	✓ Concluído
2	Desenvolvimento do site (HTML/CSS/JS)	✓ Concluído
3	Integração com GitHub API	✓ Concluído

Fase	Etapas	Status
4	Configuração de segurança	✔ Concluído
5	Deploy via GitHub Pages	✔ Concluído
6	Documentação e apresentação	✔ Concluído

1.4 Stack Tecnológico Selecionado

Componente	Tecnologia	Justificativa
Frontend	HTML5, CSS3, JavaScript (ES6+)	Sem necessidade de framework para site estático
Hospedagem	GitHub Pages	Integração nativa com repositório, HTTPS automático
API de Dados	GitHub REST API v3	Dados reais e atualizados diretamente da fonte
Ícones	Font Awesome 6.4.0	Biblioteca confiável com CDN oficial
Versionamento	Git + GitHub	Controle de versão, histórico e colaboração
IA de Suporte	Google Gemini	Guia de implantação, validação de decisões técnicas

2. Configuração Inicial e Integração com GitHub

2.1 Configuração do Repositório

O repositório **Bootcamp-Final** foi criado no GitHub com as seguintes configurações iniciais:

```
Nome:          Bootcamp-Final
Visibilidade:  Public
Branch padrão: main
GitHub Pages:  Ativado (source: branch main, raiz /)
Descrição:     Entrega Final - PortfolioHUB + GitHub API Integration
```

Estrutura de arquivos do repositório:

```
Bootcamp-Final/
├── index.html          - Estrutura HTML do portfólio
├── style.css           - Estilização e tema visual
├── script.js           - Lógica JavaScript + integração GitHub API
├── SECURITY.md          - Política de segurança do repositório
├── DOCUMENTACAO.md     - Este documento
├── README.md           - Documentação principal do repositório
├── pessoas/
│   └── BJ-sim/
│       ├── Blackjack-Simulator.py - Projeto pessoal: simulador
│       └── README.md
```

2.2 Integração com a GitHub REST API

A integração com o GitHub foi implementada diretamente no `script.js` utilizando a **GitHub REST API v3** (endpoint público, sem necessidade de autenticação para dados públicos).

Endpoints utilizados:

Endpoint	Dados obtidos
<code>GET /users/VitorMuzzi</code>	Nome, avatar, bio, contagem de repos, seguidores
<code>GET /users/VitorMuzzi/repos?sort=updated&per_page=6</code>	6 repositórios mais recentes com metadados

Trecho do código de integração (`script.js`):

```
const GITHUB_USERNAME = 'VitorMuzzi';

async function initGitHub() {
  const [userRes, reposRes] = await Promise.all([
    fetch(`https://api.github.com/users/${GITHUB_USERNAME}`),
    fetch(`https://api.github.com/users/${GITHUB_USERNAME}/repos?sort=updated&per_page=6`)
  ]);
  const user = await userRes.json();
  const repos = await reposRes.json();
  renderGitHubProfile(user);
  renderGitHubRepos(repos);
}
```

Funcionalidades da integração:

- **Perfil dinâmico:** Avatar, nome real, bio e estatísticas carregados da API em tempo real
- **Grade de repositórios:** 6 repositórios mais recentes exibidos automaticamente com nome, descrição, linguagem (com cor correspondente à paleta do GitHub), estrelas, forks e tempo de última atualização
- **Tratamento de erros:** Estado de fallback exibido se a API não responder (ex: limite de rate atingido)
- **Loading state:** Animação de carregamento enquanto os dados são buscados

2.3 Funcionalidades do Site

O PortfolioHUB implementa as seguintes seções:

1. **Header/Hero:** Nome, cargo, subtítulo com efeito de digitação animado, botões de ação
2. **Sobre Mim:** Texto descritivo sobre formação e objetivos profissionais
3. **Stack Tecnológico:** Cards clicáveis de habilidades com modais explicativos para cada tecnologia
4. **Projetos em Destaque:** Cards dos projetos com modal detalhado, link para repositório e galeria de imagens
5. **GitHub (novo):** Seção dinâmica com perfil e grade de repositórios carregados via API
6. **Footer:** Links sociais e créditos

3. Gestão de Usuários e Segurança

3.1 Gestão de Usuários no GitHub

Configuração da conta:

Configuração	Status	Detalhe
Autenticação 2FA	✓ Ativa	Autenticador via app (Google Authenticator)
E-mail verificado	✓ Sim	E-mail principal confirmado
Nome público	✓ Definido	Vitor Gomes Muzzi
Foto de perfil	✓ Definida	Avatar sincronizado no portfólio via API

Níveis de acesso no repositório:

Papel	Permissão	Usuários
Owner (Admin)	Leitura, escrita, configuração	VitorMuzzi
Collaborator	Leitura apenas	Nenhum atualmente
Público	Leitura (fork/clone)	Qualquer pessoa

3.2 Políticas de Segurança Implementadas

Branch Protection Rules (main):

- ✓ Require pull request before merging
- ✓ Require at least 1 approving review
- ✓ Dismiss stale pull request approvals when new commits are pushed
- ✓ Require branches to be up to date before merging
- ✓ Do not allow bypassing the above settings

GitHub Security Features ativadas:

Feature	Status	Objetivo
Dependabot security alerts	✓ Ativo	Detecta vulnerabilidades em dependências
Dependabot version updates	✓ Ativo	Atualiza dependências automaticamente
Secret scanning	✓ Ativo	Previne exposição acidental de tokens/chaves
Code scanning (CodeQL)	✓ Ativo	Análise estática de código

Política de Segurança do Código:

- Nenhum token, senha ou chave de API no código-fonte
- A integração com a GitHub API usa apenas endpoints públicos, sem autenticação client-side

- Sem coleta de dados de usuários visitantes
- Sem cookies ou localStorage com dados sensíveis
- Arquivo `SECURITY.md` criado documentando o processo de reporte de vulnerabilidades

3.3 Validação com Google Gemini

O Gemini foi consultado para validação das políticas de segurança implementadas:

Prompt: "Analise as seguintes configurações de segurança para um repositório público no GitHub e avalie se estão alinhadas com as melhores práticas: [lista das configurações]."

O Gemini confirmou a conformidade com as melhores práticas e sugeriu adicionalmente:

- Adicionar um arquivo `CODEOWNERS` para projetos com múltiplos colaboradores
- Configurar alertas de email para eventos de segurança — **implementado** nas configurações de notificação da conta

4. Compartilhamento e Controle de Acesso com GitHub

4.1 Modelo de Controle de Versão

O repositório segue o modelo de **trunk-based development** simplificado, adequado para projeto individual:

```
main (branch protegida)
├─ feature/* (branches para novas funcionalidades)
│   └─ Pull Request → revisão → merge para main
```

Histórico de commits relevantes:

Descrição do commit	Impacto
init: estrutura inicial do portfólio	Base HTML/CSS/JS
feat: seção de habilidades com modais	Interatividade de skills
feat: cards de projetos com galeria	Projetos Kyndo e BJ-Sim
feat: integração com GitHub API	Seção GitHub dinâmica
fix: bug de navegação por teclado no modal	Correção do indexOf em src URLs
docs: SECURITY.md e documentação final	Segurança e documentação

4.2 Configuração do GitHub Pages

O deploy automático foi configurado via **GitHub Pages**:

```
Source:  Deploy from branch
Branch:  main
Folder:  / (root)
```

```
HTTPS: Enforced
URL: https://vitormuzzi.github.io/Bootcamp-Final/
```

A cada push para `main`, o GitHub Pages reconstrói e publica automaticamente o site — sem necessidade de pipeline de CI adicional para projetos estáticos.

4.3 Práticas de Colaboração Documentadas

Workflow de contribuição:

1. Fork ou criação de branch `feature/nome-da-feature`
2. Desenvolvimento local com commits semânticos (`feat:`, `fix:`, `docs:`, `refactor:`)
3. Pull Request aberto com descrição clara das mudanças
4. Revisão de código (pelo próprio autor ou colaborador)
5. Merge squash para `main` após aprovação
6. Deploy automático via GitHub Pages

Convenção de commits adotada (Conventional Commits):

```
feat: Nova funcionalidade
fix: Correção de bug
docs: Documentação
style: Formatação sem mudança de lógica
refactor: Refatoração sem mudança de comportamento
chore: Tarefas de manutenção
```

4.4 Compartilhamento e Acesso Público

O portfólio é **intencionalmente público**, servindo como vitrine profissional:

- O código-fonte é aberto para leitura e referência
- O site é acessível publicamente em HTTPS
- A integração com a API do GitHub torna os dados do perfil sempre atualizados automaticamente
- Qualquer atualização de projetos no GitHub reflete automaticamente no portfólio (repos recentes)

5. Finalização da Integração e Testes

5.1 Checklist de Funcionalidades Testadas

Funcionalidade	Resultado
Carregamento da página	✅ OK
Efeito de digitação no header	✅ OK
Navegação âncora (Sobre, Habilidades, Projetos, GitHub)	✅ OK
Modal de tecnologias (clique em cada skill tag)	✅ OK

Funcionalidade	Resultado
Modal de projetos — Kyndo	✓ OK
Modal de projetos — Blackjack Simulator	✓ OK
Botão "Repositório" no modal (link externo)	✓ OK
Fechamento de modal (botão X)	✓ OK
Fechamento de modal (clique fora)	✓ OK
Fechamento de modal (tecla Esc não implementada)	⚠ Melhoria futura
Seção GitHub — carregamento de perfil via API	✓ OK
Seção GitHub — exibição de avatar	✓ OK
Seção GitHub — contadores (repos/seguidores)	✓ OK
Seção GitHub — grade de repositórios	✓ OK
Seção GitHub — cores de linguagem	✓ OK
Seção GitHub — link para cada repositório	✓ OK
Seção GitHub — estado de erro (API indisponível)	✓ OK
Responsividade mobile (< 768px)	✓ OK
GitHub Pages — HTTPS ativo	✓ OK

5.2 Bugs Corrigidos Durante o Projeto

Bug 1 — Card do Blackjack com dados placeholder:

Antes: `openProjectModal('nome do repositorio', [], 'descrição do projeto', ...)` *Depois:* Título, descrição completa, link correto para `pessoais/BJ-sim` e tecnologias (Python, Probabilidade, Hi-Lo) adicionados.

Bug 2 — Imagem inexistente no card do Kyndo:

Antes: `['image_0.png']` passado como array de imagens — arquivo não existia no repositório, gerando erro 404. *Depois:* Array vazio `[]` — galeria oculta quando não há imagens, sem erros.

Bug 3 — Navegação por teclado no modal com comportamento incorreto:

Antes: `currentProjectImages.indexOf(img.src)` — `img.src` retorna URL absoluta (`http://localhost/image_0.png`), mas o array contém caminhos relativos, então `indexOf` sempre retornava `-1`. *Depois:* Variável `currentImageIndex` rastreia o índice atual separadamente, eliminando a dependência de comparação de URLs.

Bug 4 — LinkedIn link com URL de exemplo:

Antes: `href="https://linkedin.com/in/seu-linkedin"` — URL de template inexistente. *Depois:* `href="#"` com título descritivo — inofensivo até o usuário adicionar a URL real.

5.3 Deploy em Produção

O site está disponível em produção e acessível publicamente:

URL de produção: <https://vitormuzzi.github.io/Bootcamp-Final/>

Verificações de produção:

- HTTPS ativo e certificado válido (GitHub Pages + Let's Encrypt)
- Tempo de carregamento < 2s (sem backend, assets leves)
- API do GitHub respondendo e populando a seção dinâmica
- Responsividade testada em desktop e mobile

5.4 Validação Final com Google Gemini

O Gemini foi utilizado para revisão final do código antes do deploy:

Prompt: "Revise este trecho de JavaScript que faz fetch da API do GitHub e renderiza repositórios dinamicamente. Há algum problema de segurança, performance ou tratamento de erro que devo adicionar?"

O Gemini identificou e confirmou que:

- O tratamento de erro com `try/catch` e estado de fallback está correto
- O uso de `Promise.all` para requests paralelos é a abordagem recomendada
- Não há exposição de tokens ou dados sensíveis no código client-side
- A truncagem de descrições longas previne overflow de layout

6. Revisão Final e Apresentação

6.1 Resumo das Etapas Realizadas

Etapa	Descrição	Resultado
1	Planejamento com apoio do Google Gemini	Plano de 6 fases definido e seguido
2	Criação e configuração do repositório GitHub	Repo com GitHub Pages, branch protection e security features
3	Desenvolvimento do site PortfolioHUB	Site responsivo com 5 seções funcionais
4	Integração com GitHub API	Seção dinâmica com perfil e repositórios em tempo real
5	Configuração de segurança	SECURITY.md, 2FA, branch protection, Dependabot, Secret Scanning
6	Testes e documentação	Todos os bugs corrigidos, funcionalidades validadas

6.2 Soluções Implementadas e Desafios Superados

Desafio 1 — Integração da API sem backend: A GitHub REST API tem limite de 60 requisições/hora para chamadas não autenticadas. A solução foi usar `Promise.all` para fazer apenas 2 chamadas simultâneas (user + repos), minimizando o consumo do limite e garantindo dados atualizados a cada visita.

Desafio 2 — Dados dinâmicos em site estático: Sem servidor para pré-renderizar dados, a seção GitHub carrega de forma assíncrona com estado de loading visível ao usuário. Se a API falhar, um estado de erro amigável é exibido com link direto para o perfil do GitHub.

Desafio 3 — Segurança em repositório público: Como o código é público, qualquer token no código-fonte seria imediatamente exposto. A solução foi usar exclusivamente endpoints públicos da API, que não requerem autenticação para dados de perfil e repositórios públicos.

Desafio 4 — Bug de navegação por teclado: O uso de `img.src` para buscar o índice atual na array de imagens falhava silenciosamente porque o browser converte caminhos relativos para URLs absolutas. A solução foi rastrear o índice com uma variável dedicada (`currentImageIndex`).

6.3 Tecnologias e Ferramentas Utilizadas no Projeto

- **HTML5/CSS3/JavaScript ES6+:** Construção e estilização do site
- **GitHub REST API v3:** Integração dinâmica de dados de perfil e repositórios
- **GitHub Pages:** Hospedagem gratuita com HTTPS automático
- **Git:** Versionamento com commits semânticos
- **Google Gemini:** Guia de planejamento, validação de segurança e revisão de código
- **Font Awesome 6.4.0:** Biblioteca de ícones

6.4 Link para Apresentação em Vídeo

<https://youtu.be/qUnUF3alhyl>

O vídeo de apresentação cobre:

1. Demonstração do site em produção (<https://vitormuzzi.github.io/Bootcamp-Final/>)
2. Navegação pelas seções: Header, Sobre, Habilidades, Projetos, GitHub
3. Demonstração da integração em tempo real com a GitHub API
4. Breve walkthrough do código de integração no `script.js`
5. Exibição das configurações de segurança no GitHub (Branch Protection, Dependabot, SECURITY.md)
6. Discussão dos desafios superados e soluções implementadas

Documento gerado em: 14/06/2026 Repositório: <https://github.com/VitorMuzzi/Bootcamp-Final>